

[← Back to Tech Articles](#)

Understanding jemalloc: A Rust Performance Booster

Mar 16, 2025

Rust

Ethan Miller

Product Engineer · Leapcell

This post is also available in [日本語](#), [한국어](#), and [Deutsch](#)



jemalloc

What is jemalloc?

jemalloc is a modern memory allocator originally developed by Jason Evans for FreeBSD. Compared to traditional `malloc`, jemalloc was designed from the outset to reduce memory fragmentation and enhance performance in multithreaded applications. It achieves fast and efficient memory allocation



We're Leapcell, the next-gen web hosting platform that can:

20 = \$0

Deploy 20 projects for **FREE**. No charge for idle machines.



Host any Python, Go, Rust or Node.js projects.

Deploy on Leapcell for **FREE!**

Popular Posts

Why Use Underscore Fields in Go Structs

Why Modern Languages (Go, Rust) Prefer Composition Over Inheritance

Mastering Go Testing

Your Rust Is Too Slow - 20 Practical Ways to Optimize Your Code

[More Posts →](#)

through fine-grained memory management and thread caching mechanisms.

The main features of jemalloc include:

- **Efficient Concurrent Processing:** By providing independent memory caches for each thread, it reduces contention between threads.
- **Reduced Memory Fragmentation:** Its carefully designed memory allocation strategy improves memory utilization.
- **Cross-Platform Support:** It supports multiple operating systems, including Linux, macOS, and Windows.

Traditional malloc

- **Definition:** `malloc` is the most basic memory allocation function, providing a simple way to allocate a block of memory of a specified size. It is part of the C standard library and is widely used in C and C++ programs.
- **How It Works:** When a program calls `malloc` to request memory, `malloc` searches for a sufficiently large continuous memory region in the process's heap space to allocate to the program. If the request is successful, `malloc` returns a pointer to the allocated memory; if it fails, it returns `NULL`.
- **Characteristics:** The implementation of `malloc` focuses on generality rather than performance optimization in specific scenarios. As a result, it may exhibit performance bottlenecks in handling numerous small allocations or in multithreaded environments, such as memory fragmentation or lock contention.

jemalloc

- **Definition:** jemalloc is a modern memory allocator developed by Jason Evans, aimed at providing high-performance and low-fragmentation memory management. Initially designed for the FreeBSD operating

system, it is now widely used in other systems and major projects, such as Facebook's production environment and the Rust programming language's standard library (although Rust later reverted to using the system allocator by default).

- **How It Works:** jemalloc employs various strategies to optimize memory allocation and reclamation, such as using size classes to manage allocations of different sizes, thread caching to reduce lock contention, and delayed reclamation and segmentation strategies to minimize memory fragmentation.
- **Characteristics:** jemalloc is designed to enhance the performance of multithreaded applications and reduce memory fragmentation. Its finely-tuned memory management strategies and optimized data structures make it particularly suitable for scenarios that involve handling large volumes of concurrent memory operations.

Comparison

- **Performance:** jemalloc typically performs better in highly concurrent and memory allocation-intensive applications, especially in reducing memory fragmentation and improving allocation efficiency.
- **Usage Scenarios:** While `malloc` is available in all standard C environments, developers may choose jemalloc or other optimized memory allocators for scenarios that demand higher performance.
- **Compatibility:** jemalloc can be used as a direct replacement for `malloc`, and in most cases, existing code does not need modification to benefit from its performance advantages.

Why Use jemalloc in Rust?

Although Rust's standard library reverted to using the system allocator by default starting from version 1.32, jemalloc remains the preferred choice in high-performance application scenarios. This is especially true for applications involving extensive concurrent memory operations, such as high-concurrency web servers, database systems, and blockchain technologies. Using jemalloc can significantly enhance performance and memory usage efficiency in these contexts.

Using jemalloc in Rust

To use jemalloc in a Rust project, you can add the `jemallocator` crate as a dependency. Here's how to configure it:

1. Add Dependencies

First, add the `jemallocator` crate in your `Cargo.toml` file:

```
[
dependencies
]
jemallocator = "0.3"
```

2. Configure the Global Allocator

Next, configure jemalloc as the global allocator in the entry point of your Rust application (such as `main.rs` or the root file of your library `lib.rs`):

```
extern crate jemallocator;

#[global_allocator]
static GLOBAL: jemallocator::Jemalloc = jemallocator::Jemalloc;
```

This code declares jemalloc as the global memory allocator. Through this method, all memory allocations in the Rust program will be handled by jemalloc.

3. Compile and Run Your Program

After the configuration is complete, compile and run your Rust program as usual. If everything is set up correctly, your program will now use jemalloc as its memory allocator.

Conclusion

Overall, jemalloc provides an efficient memory management method, particularly suitable for Rust applications that require high concurrency and high memory efficiency. Although Rust defaults to using the system allocator, switching to jemalloc in scenarios with specific performance demands can bring significant performance improvements to Rust applications, such as backend services and blockchain projects.

[We are Leapcell, your top choice for hosting Rust projects.](#)



Leapcell

Serverless Web Hosting / Redis / Async Job

[Leapcell](#) is the Next-Gen Serverless Platform for Web Hosting, Async Tasks, and Redis:

Multi-Language Support

- Develop with Node.js, Python, Go, or Rust.

Deploy unlimited projects for free

- pay only for usage — no requests, no charges.

Unbeatable Cost Efficiency

- Pay-as-you-go with no idle charges.
- Example: \$25 supports 6.94M requests at a 60ms average response time.

Streamlined Developer Experience

- Intuitive UI for effortless setup.
- Fully automated CI/CD pipelines and GitOps integration.
- Real-time metrics and logging for actionable insights.

Effortless Scalability and High Performance

- Auto-scaling to handle high concurrency with ease.
- Zero operational overhead — just focus on building.

Explore more in the [Documentation!](#)



Follow us on X: [@LeapcellHQ](#)

 Share this article



More Posts from Leapcell

